

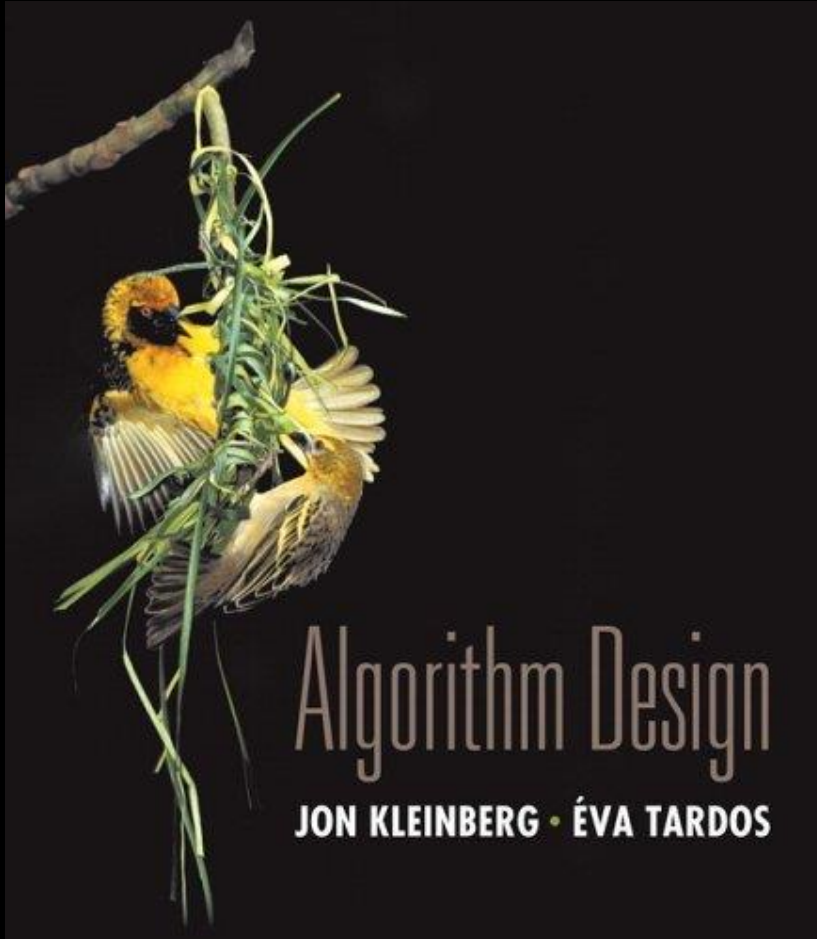
Course evaluation

<https://evaluation.shanghaitech.edu.cn/tqnmaep/login>



Chapter 13

Randomized Algorithms



Slides by Kevin Wayne.
Copyright © 2005 Pearson-Addison Wesley.
All rights reserved.

13.4 MAX 3-SAT

Maximum 3-Satisfiability

MAX-3SAT. Given 3-SAT formula, find a truth assignment that satisfies as many clauses as possible.

$$\begin{aligned}C_1 &= x_2 \vee \overline{x_3} \vee \overline{x_4} \\C_2 &= x_2 \vee x_3 \vee \overline{x_4} \\C_3 &= \overline{x_1} \vee x_2 \vee x_4 \\C_4 &= \overline{x_1} \vee \overline{x_2} \vee x_3 \\C_5 &= x_1 \vee \overline{x_2} \vee \overline{x_4}\end{aligned}$$

Remark. NP-hard search problem.

Simple idea. Flip a coin, and set each variable true with probability $\frac{1}{2}$, independently for each variable.

Maximum 3-Satisfiability: Analysis

Claim. Given a 3-SAT formula with k clauses, the **expected number** of clauses satisfied by a random assignment is $7k/8$.

Pf. Consider random variable $Z_j = \begin{cases} 1 & \text{if clause } C_j \text{ is satisfied} \\ 0 & \text{otherwise.} \end{cases}$

▫ Let Z = number of clauses satisfied.

$$\begin{aligned} E[Z] &= \sum_{j=1}^k E[Z_j] \\ \text{linearity of expectation} \quad &\nearrow \\ &= \sum_{j=1}^k \Pr[\text{clause } C_j \text{ is satisfied}] \\ &= \frac{7}{8}k \end{aligned}$$

The Probabilistic Method

Corollary. For any instance of 3-SAT, **there exists** a truth assignment that satisfies at least a $7/8$ fraction of all clauses.

Pf. Random variable is at least its expectation some of the time. ▪

Maximum 3-Satisfiability: Analysis

Q. Can we turn this idea into a $7/8$ -approximation algorithm? In general, a random variable can almost always be below its mean.

Lemma. The probability that a random assignment satisfies $\geq 7k/8$ clauses is at least $1/(8k)$.

Pf. Let p_j be probability that exactly j clauses are satisfied; let p be probability that $\geq 7k/8$ clauses are satisfied.

$$\begin{aligned}\frac{7}{8}k &= E[Z] = \sum_{j \geq 0} j p_j \\ &= \sum_{j < 7k/8} j p_j + \sum_{j \geq 7k/8} j p_j \\ &\leq \left(\frac{7k}{8} - \frac{1}{8}\right) \sum_{j < 7k/8} p_j + k \sum_{j \geq 7k/8} p_j \\ &\leq \left(\frac{7}{8}k - \frac{1}{8}\right) \cdot 1 + k p\end{aligned}$$

Rearranging terms yields $p \geq 1 / (8k)$. ▀

Maximum 3-Satisfiability: Analysis

Johnson's algorithm. Repeatedly generate random truth assignments until one of them satisfies $\geq 7k/8$ clauses.

Theorem. Johnson's algorithm is a $7/8$ -approximation algorithm with polynomial **expected** running time.

Pf. By previous lemma, each iteration succeeds with probability at least $1/(8k)$. By the waiting-time bound, the expected number of trials to find the satisfying assignment is at most $8k$. ▀

Two Types of Randomized Algorithms

Two Algorithms

Contraction algorithm for global min cut

- $\Omega(m)$ running time
- Returns a min cut **with prob** $\geq 2/n^2$

Johnson's algorithm for MAX-3SAT

- Guarantees 7/8-approximation (by repeating until success)
- Polynomial **expected** running time

Monte Carlo vs. Las Vegas Algorithms

Monte Carlo algorithm. Guaranteed running time, likely to find correct answer.

Ex: Contraction algorithm for global min cut.

Las Vegas algorithm. Guaranteed to find correct answer, likely to run in certain time.

Ex: Randomized quicksort, Johnson's MAX-3SAT algorithm.



Monte Carlo vs. Las Vegas Algorithms

Monte Carlo algorithm. Guaranteed running time, likely to find correct answer.

Ex: Contraction algorithm for global min cut.

Las Vegas algorithm. Guaranteed to find correct answer, likely to run in certain time.

Ex: Randomized quicksort, Johnson's MAX-3SAT algorithm.

stop algorithm after a certain point



Remark. Can always convert a Las Vegas algorithm into Monte Carlo, but no known method to convert the other way.

RP and ZPP

RP: (Randomized Polynomial) time. [Monte Carlo] Decision problems solvable with **one-sided error** in poly-time.

One-sided error.

Can decrease probability of false negative to 2^{-100} by 100 independent repetitions

- If the correct answer is **no**, always return **no**.
- If the correct answer is **yes**, return **yes** with probability $\geq \frac{1}{2}$.

ZPP: (Zero-error Probabilistic Polynomial) time. [Las Vegas] Decision problems solvable in **expected** poly-time.

↑
running time can be unbounded, but on average it is fast

Theorem. $P \subseteq ZPP \subseteq RP \subseteq NP$.

Fundamental open questions. To what extent does randomization help?
Does $P = ZPP$? Does $ZPP = RP$? Does $RP = NP$?

13.5 Randomized Divide-and-Conquer

Quicksort

Quicksort

- Pick one element to use as pivot.
- Partition elements into two sub-arrays:
 - Elements less than or equal to pivot
 - Elements greater than pivot
- Quicksort two sub-arrays

40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

Select pivot

20	10	7	30	40	80	60	50	100
----	----	---	----	----	----	----	----	-----

Partition

$\leq$ pivot

$>$ pivot

7	10	20	30	40	50	60	80	100
---	----	----	----	----	----	----	----	-----

Quicksort sub-arrays

Quicksort

Randomized Quicksort

- Pick one element to use as pivot **uniformly at random**.
- Partition elements into two sub-arrays:
 - Elements less than or equal to pivot
 - Elements greater than pivot
- Quicksort two sub-arrays

40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

Select pivot

20	10	7	30	40	80	60	50	100
----	----	---	----	----	----	----	----	-----

Partition

$\leq$ pivot

$>$ pivot

7	10	20	30	40	50	60	80	100
---	----	----	----	----	----	----	----	-----

Quicksort sub-arrays

Quicksort

Running time.

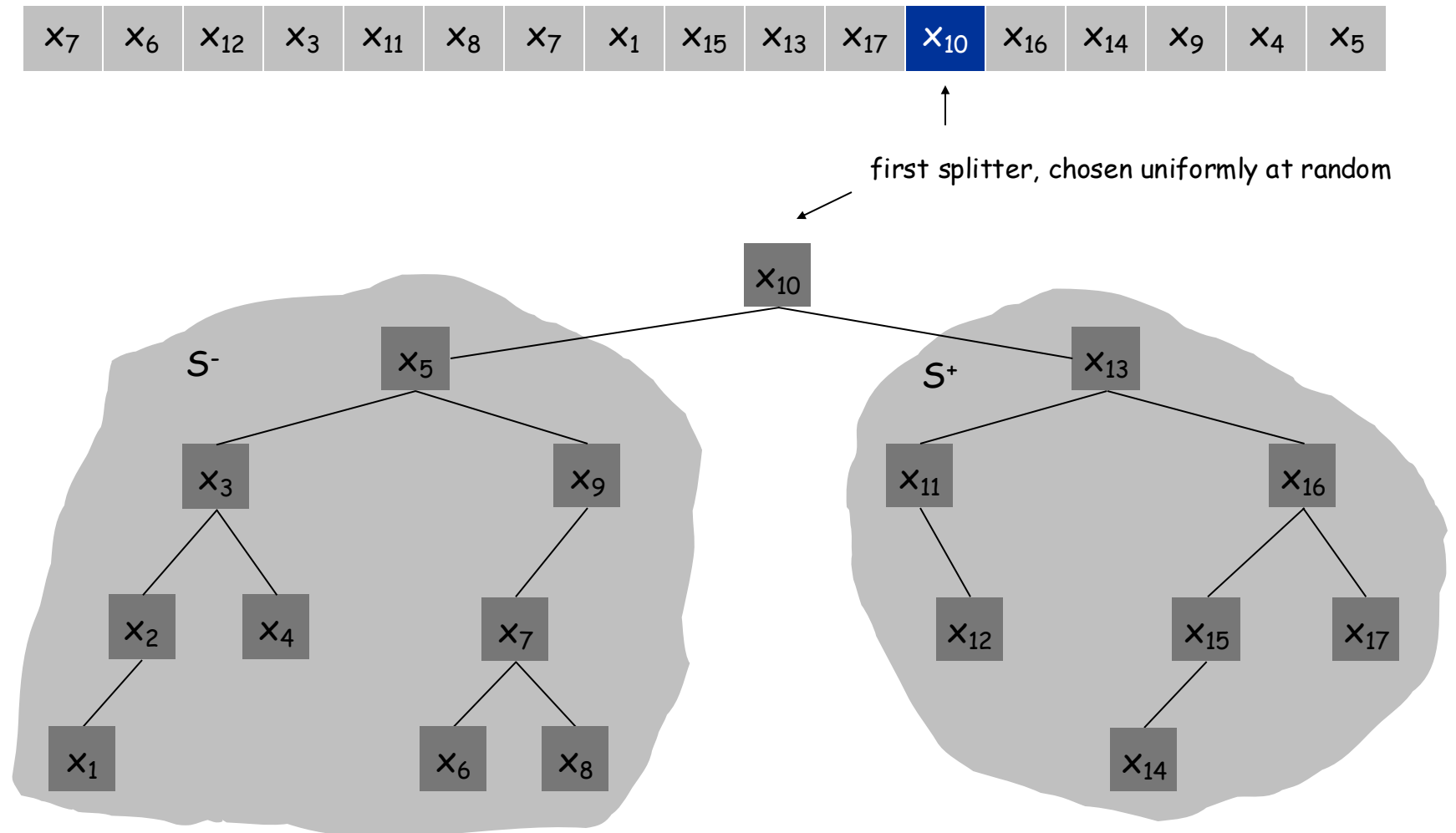
- [Best case.] Select the median element as the splitter: quicksort makes $\Theta(n \log n)$ comparisons.
- [Worst case.] Select the smallest/largest element as the splitter: quicksort makes $\Theta(n^2)$ comparisons.

Randomize. Protect against worst case by choosing splitter at **random**.

Quicksort: BST Representation of Splitters

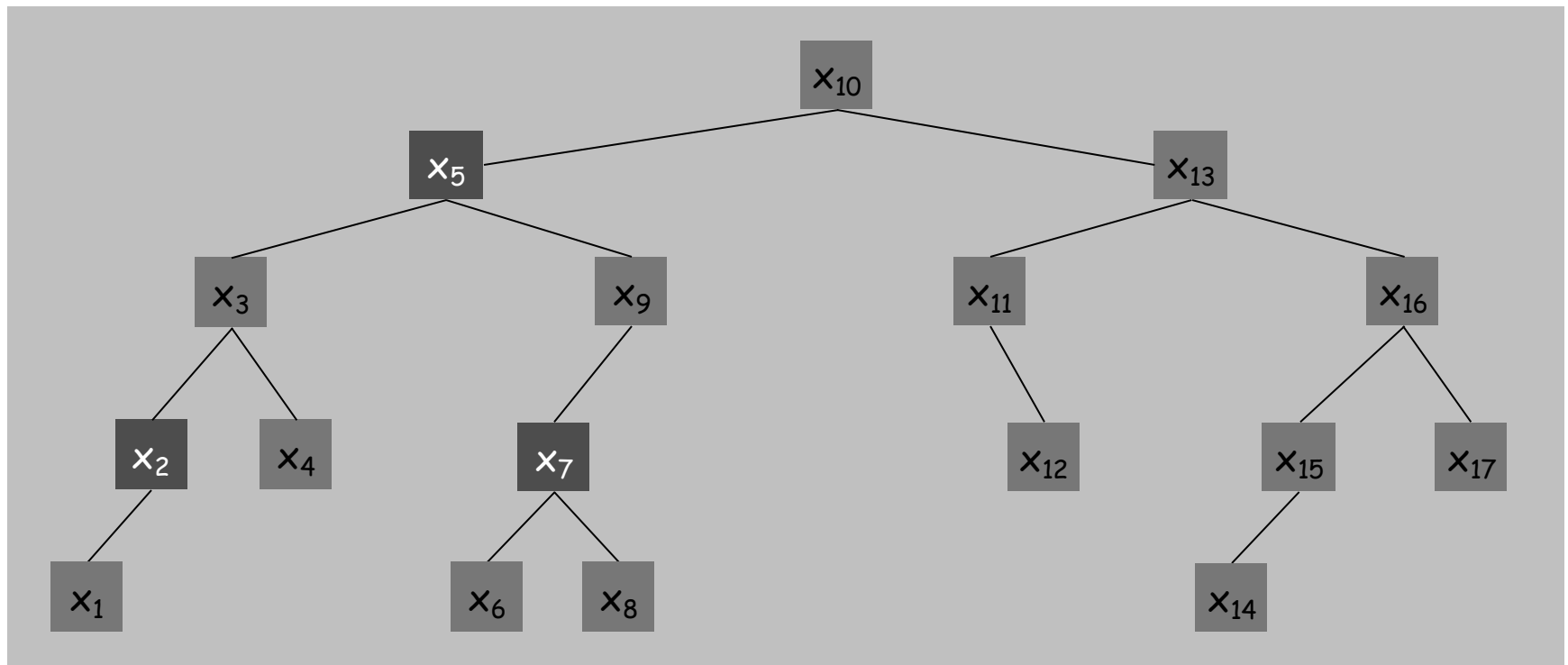
Notation. Label elements so that $x_1 < x_2 < \dots < x_n$.

BST representation. Draw recursive BST of splitters.



Quicksort: BST Representation of Splitters

Observation. Element only compared with its ancestors and descendants.

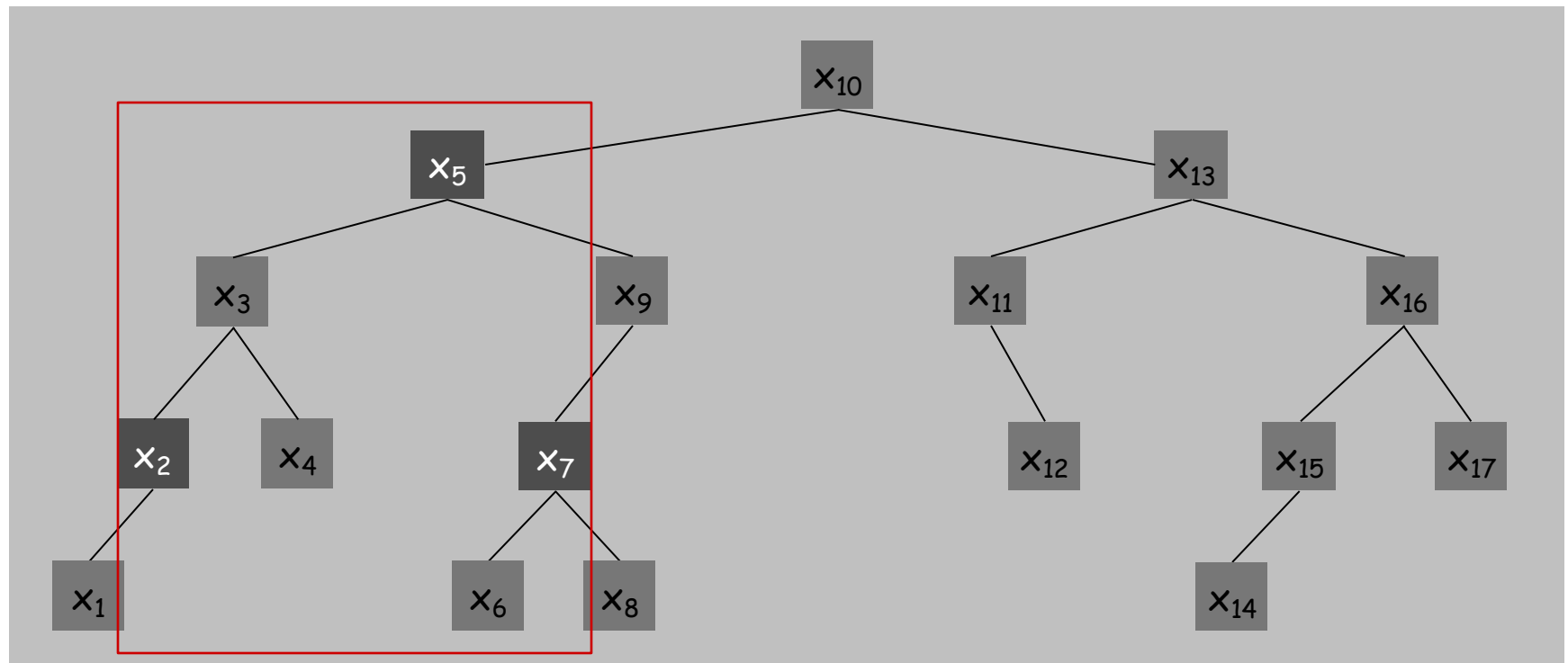


Quicksort: BST Representation of Splitters

Claim. $\Pr[x_i \text{ and } x_j \text{ are compared}, i < j] = 2 / (j - i + 1)$.

Proof:

- The lca of x_i and x_j must be x_k where $i \leq k \leq j$
- $x_i, \dots, x_j$ must belong to the subtree rooted at x_k
- The first pivot among $x_i, \dots, x_j$ is x_k
- x_i and x_j are compared if and only if $x_k = x_i$ or x_j .



Quicksort: Expected Number of Comparisons

Theorem. Expected # of comparisons is $O(n \log n)$.

Pf.

$$E[C] = \sum_{1 \leq i < j \leq n} E[C_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j-i+1} = 2 \sum_{j=2}^n \sum_{k=2}^j \frac{1}{k} \leq 2n \sum_{k=2}^n \frac{1}{k} = 2nH(n)$$

$k = j - i + 1$
↓
↑
probability that i and j are compared

$\ln(n+1) < H(n) < 1 + \ln n$
↑

Theorem. [Knuth 1973] Stddev of number of comparisons is $\sim 0.65n$.

Ex. If $n = 1$ million, the probability that randomized quicksort takes less than $4n \ln n$ comparisons is at least 99.94%.

Chebyshev's inequality. $\Pr[|X - \mu| \geq k\sigma] \leq 1 / k^2$.

Chapter Summary

Basics in Probability Theory

Union bound. Given events $E_1, \dots, E_n$, $\Pr\left[\bigcup_{i=1}^n E_i\right] \leq \sum_{i=1}^n \Pr[E_i]$

Waiting time expectation. Expected number of trials for a first success is $1/p$.

Linearity of expectation. Given two random variables X and Y defined over the same probability space, $E[X + Y] = E[X] + E[Y]$.

Algorithms

Monte Carlo algorithm. Guaranteed running time, likely to find correct answer.

- Contraction algorithm for global min cut

Las Vegas algorithm. Guaranteed to find correct answer, likely to run in certain time.

- Johnson's MAX-3SAT algorithm
- Randomized quicksort